

Compilation

Typage

Adrien Guatto

Master 1 Informatique
Semestre 1 – 2025

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Pourquoi le typage? Retour au premier évaluateur L1

```
type value = VInt of int | VFun of bound1
```

```
let rec eval : t -> value = function
| Var _ -> invalid_arg "open expression"
| Int n -> VInt n
| Add (e1, e2) -> add_values (eval e1) (eval e2)
| Let (e, b) -> eval (close_with b (reify (eval e)))
| Fun b -> VFun b
| App (e1, e2) -> app_values (eval e1) (eval e2)
```

```
and add_values v1 v2 = match v1, v2 with
| VInt n1, VInt n2 -> VInt (n1 + n2)
| _ -> failwith "ill-typed"
```

```
and app_values v1 v2 = match v1 with
| VFun b -> eval (close_with b (reify v2))
| _ -> failwith "ill-typed"
```

Pourquoi le typage ? Retour au premier évaluateur L1

```
type value = VInt of int | VFun of bound1
```

```
let rec eval : t -> value = function
| Var _ -> invalid_arg "open expression"
| Int n -> VInt n
| Add (e1, e2) -> add_values (eval e1) (eval e2)
| Let (e, b) -> eval (close_with b (reify (eval e)))
| Fun b -> VFun b
| App (e1, e2) -> app_values (eval e1) (eval e2)
```

```
and add_values v1 v2 = match v1, v2 with
| VInt n1, VInt n2 -> VInt (n1 + n2)
| _ -> failwith "ill-typed"
```

```
and app_values v1 v2 = match v1 with
| VFun b -> eval (close_with b (reify v2))
| _ -> failwith "ill-typed"
```

Comment s'assurer que ce code ne lève pas **Failure** "ill-typed" ?

Les systèmes de types

Qu'est-ce qu'un système de types ?

- Un jeu de règles syntaxiques pour associer à une expression un **type**.
- Un type est une petite formule logique qui décrit une **propriété**.
- Un programme **bien typé** satisfait la propriété décrite par son type.

Les systèmes de types

Qu'est-ce qu'un système de types ?

- Un jeu de règles syntaxiques pour associer à une expression un **type**.
- Un type est une petite formule logique qui décrit une **propriété**.
- Un programme **bien typé** satisfait la propriété décrite par son type.

Quelques exemples de propriétés que l'on peut garantir par typage :

- OCAML : absence de corruption mémoire ;

Les systèmes de types

Qu'est-ce qu'un système de types ?

- Un jeu de règles syntaxiques pour associer à une expression un **type**.
- Un type est une petite formule logique qui décrit une **propriété**.
- Un programme **bien typé** satisfait la propriété décrite par son type.

Quelques exemples de propriétés que l'on peut garantir par typage :

- OCAML : absence de corruption mémoire ;
- C : données manipulées en accord avec leurs tailles (**sizeof**) ;

Les systèmes de types

Qu'est-ce qu'un système de types ?

- Un jeu de règles syntaxiques pour associer à une expression un **type**.
- Un type est une petite formule logique qui décrit une **propriété**.
- Un programme **bien typé** satisfait la propriété décrite par son type.

Quelques exemples de propriétés que l'on peut garantir par typage :

- OCAML : absence de corruption mémoire ;
- C : données manipulées en accord avec leurs tailles (**sizeof**) ;
- ROCQ : énoncé mathématique arbitraire (cf. *assistants de preuve*, S2) ;

Les systèmes de types

Qu'est-ce qu'un système de types ?

- Un jeu de règles syntaxiques pour associer à une expression un **type**.
- Un type est une petite formule logique qui décrit une **propriété**.
- Un programme **bien typé** satisfait la propriété décrite par son type.

Quelques exemples de propriétés que l'on peut garantir par typage :

- OCAML : absence de corruption mémoire ;
- C : données manipulées en accord avec leurs tailles (**sizeof**) ;
- ROCQ : énoncé mathématique arbitraire (cf. *assistants de preuve*, S2) ;

Terminologie douteuse et pop-culture informatique

- “typage dynamique” : on parlera plutôt de langage **non-typé**.
- “typage fort, typage faible” : à bannir, n'a pas de sens précis.

Cette séance de cours

Contenu

- Introduction aux différents concepts des systèmes de types.
 - Sous la forme de code OCaml et de formalisme mathématique.
- Deux systèmes de types et les extensions linguistiques attenantes.
 - Types simples (monomorphes).
 - Types polymorphes.
- Une technique d'implémentation simple, le typage bidirectionnel.

Cette séance de cours

Contenu

- Introduction aux différents concepts des systèmes de types.
 - Sous la forme de code OCaml et de formalisme mathématique.
- Deux systèmes de types et les extensions linguistiques attenantes.
 - Types simples (monomorphes).
 - Types polymorphes.
- Une technique d'implémentation simple, le typage bidirectionnel.

Un point important qui ne sera pas traité

L'inférence de types polymorphes à la ML ; voir le cours PFA au S2.

Cette séance de cours

Contenu

- Introduction aux différents concepts des systèmes de types.
 - Sous la forme de code OCaml et de formalisme mathématique.
- Deux systèmes de types et les extensions linguistiques attenantes.
 - Types simples (monomorphes).
 - Types polymorphes.
- Une technique d'implémentation simple, le typage bidirectionnel.

Un point important qui ne sera pas traité

L'inférence de types polymorphes à la ML ; voir le cours PFA au S2.

Un objectif pratique : vous préparer à la réalisation du jalon 3.

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Les types simples

- Une valeur de $L1$ est soit un entier, soit une fonction.
- Les types classifient les valeurs, donc deux constructeurs de types.

Les types simples

- Une valeur de L_1 est soit un entier, soit une fonction.
- Les types classifient les valeurs, donc deux constructeurs de types.

```
type ty = TInt | TFun of ty * ty
```

```
module Typechecker : sig  
  exception Ill_typed of ...
```

```
  (** [synth_exp e] calcule le type de l'expression [e], ou bien  
      lève l'exception {! Ill_typed} si [e] est mal typée. *)
```

```
  val synth_exp : Lang2.exp -> ty
```

```
  (** [check_exp expected e] lève l'exception {! Ill_typed} si  
      l'expression [e] n'est pas de type [expected]. *)
```

```
  val check_exp : ty -> Lang2.exp -> unit
```

```
end
```

Le module `Typechecker`, premier jet

```
exception Ill_typed of ...
```

```
let rec synth_exp = function
```

```
| Int _ -> TInt
```

```
| Add (e1, e2) -> check_exp TInt e1; check_exp TInt e2; TInt
```

```
| App (e1, e2) -> app_types (synth_exp e1) e2
```

```
| Let (e, b) -> synth_bound1 (synth_exp e) b
```

```
| Fun b -> let dom = (* ??? *) in TFun (dom, synth_bound1 dom b)
```

```
| Var x -> (* ??? *)
```

```
and check_exp expected e =
```

```
  let given = synth_exp e in
```

```
  if given <> expected then type_mismatch ~expected ~given e
```

```
and app_types ty_f e2 = match ty_f with
```

```
| TFun (dom, cod) -> check_exp dom e2; cod
```

```
| given -> type_mismatch_expected_fun given
```

```
and synth_bound1 : ty -> bound1 -> ty = fun bound_ty b -> (* ??? *)
```

Gestion des variables

- Une expression est typée dans un **contexte de typage**.
- On suppose les paramètres de fonctions annotés avec leurs types.

Gestion des variables

- Une expression est typée dans un **contexte de typage**.
- On suppose les paramètres de fonctions annotés avec leurs types.

```
type exp = ... | Fun of ty * bound1
```

```
module Typechecker : sig  
  type cx = (Id.t * ty) list
```

```
  (** [synth_exp cx e] calcule le type de l'expression [e] en  
      supposant que ses variables libres ont les types  
      prescrits par le contexte de typage [cx]. Elle lève  
      l'exception {! Ill_typed} si [e] est mal typée dans [cx]. *)
```

```
  val synth_exp : cx -> Lang2.exp -> ty
```

```
  (** [check_exp cx expected e] lève l'exception {! Ill_typed}  
      si [e] n'est pas de type [expected] dans [cx]. *)
```

```
  val check_exp : cx -> ty -> Lang2.exp -> unit
```

```
end
```

Le module `Typechecker`

```
let rec synth_exp cx = function
| Int _ -> TInt
| Add (e1, e2) -> check_exp cx TInt e1; check_exp cx TInt e2; TInt
| App (e1, e2) -> app_types cx (synth_exp e1) e2
| Let (e, b) -> synth_bound1 cx (synth_exp e) b
| Fun (dom, b) -> TFun (dom, synth_bound1 cx dom b)
| Var x -> (try List.assoc x cx with Not_found -> unbound_variable x)

and check_exp cx expected e =
  let given = synth_exp cx e in
  if given <> expected then type_mismatch ~expected ~given e

and app_types cx ty_f e2 = match ty_f with
| TFun (dom, cod) -> check_exp cx dom e2; cod
| given -> type_mismatch_expected_fun given

and synth_bound1 cx bty b = synth_exp ((b.bound, bty) :: cx) b.body
```

Sûreté du typage et remarques

Théorème (Sûreté du typage)

Si `check_exp [] e ty` termine sans erreur, il existe une valeur v telle que :

- ❶ *`eval_exp [] e = v`, en particulier sans lever d'exception,*
- ❷ *`check_val v ty` termine sans erreur.*

Sûreté du typage et remarques

Théorème (Sûreté du typage)

Si `check_exp [] e ty` termine sans erreur, il existe une valeur v telle que :

- ❶ *`eval_exp [] e = v`, en particulier sans lever d'exception,*
- ❷ *`check_val v ty` termine sans erreur.*

Remarques

- Requier une annotation de type sur tous les paramètres de fonctions.
 - De même en C, Java, Rust, etc. OCaml est capable de l'inférer.
- Un appel à `synth_exp` parcourt chaque sous-terme exactement une fois, et en particulier termine toujours ; à contraster avec l'évaluation !
`eval [] Build.(let f = fun_ (fun x -> app x [x]) in app f [f])`

Sûreté du typage et remarques

Théorème (Sûreté du typage)

Si `check_exp [] e ty` termine sans erreur, il existe une valeur v telle que :

- ❶ *`eval_exp [] e = v`, en particulier sans lever d'exception,*
- ❷ *`check_val v ty` termine sans erreur.*

Remarques

- Requier une annotation de type sur tous les paramètres de fonctions.
 - De même en C, Java, Rust, etc. OCaml est capable de l'inférer.
- Un appel à `synth_exp` parcourt chaque sous-terme exactement une fois, et en particulier termine toujours ; à contraster avec l'évaluation !
`eval [] Build.(let f = fun_ (fun x -> app x [x]) in app f [f])`

Pour démontrer le théorème, on va en général formuler le système de types dans un style mathématique plus abstrait que le code OCaml.

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Les jugements de typage

On va définir un ensemble de triplets

$$\mathcal{T} \subseteq Cx \times Expr \times Type$$

tel que $(\Gamma, e, t) \in \mathcal{T}$ lorsque e est de type t dans le contexte Γ .

Les jugements de typage

On va définir un ensemble de triplets

$$\mathcal{T} \subseteq Cx \times Expr \times Type$$

tel que $(\Gamma, e, t) \in \mathcal{T}$ lorsque e est de type t dans le contexte Γ .

L'ensemble $\mathcal{T}$ est défini comme le **plus petit** sous-ensemble de

$$Cx \times Expr \times Type$$

qui satisfait un jeu propriétés correspondant aux règles de typage.

- $\{(\Gamma, \text{Int } n, \text{int}) \mid \Gamma \in Ctx\} \subseteq \mathcal{T}$
- $\{(\Gamma, e1 + e2, \text{int}) \mid \Gamma \in Ctx, (\Gamma, e1, \text{int}) \in \mathcal{T}, (\Gamma, e2, \text{int}) \in \mathcal{T}\} \subseteq \mathcal{T}$
- $\{(\Gamma, x, t) \mid \Gamma \ni x : t\} \subseteq \mathcal{T}$
- ...

Les jugements de typage

On va définir un ensemble de triplets

$$\mathcal{T} \subseteq Cx \times Expr \times Type$$

tel que $(\Gamma, e, t) \in \mathcal{T}$ lorsque e est de type t dans le contexte Γ .

L'ensemble $\mathcal{T}$ est défini comme le **plus petit** sous-ensemble de

$$Cx \times Expr \times Type$$

qui satisfait un jeu propriétés correspondant aux règles de typage.

- $\{(\Gamma, \text{Int } n, \text{int}) \mid \Gamma \in Ctx\} \subseteq \mathcal{T}$
- $\{(\Gamma, e1 + e2, \text{int}) \mid \Gamma \in Ctx, (\Gamma, e1, \text{int}) \in \mathcal{T}, (\Gamma, e2, \text{int}) \in \mathcal{T}\} \subseteq \mathcal{T}$
- $\{(\Gamma, x, t) \mid \Gamma \ni x : t\} \subseteq \mathcal{T}$
- ...

Classiquement, l'existence de $\mathcal{T}$ est assurée par le théorème de Tarski. On peut aussi l'expliquer par une définition *inductive* (cf. parsing, jalon 2).

Une présentation inductive du jugement de typage

On écrit $\Gamma \vdash e : t$ pour $(\Gamma, e, t) \in \mathcal{T}$.

$$\begin{array}{c} \text{VAR} \\ \Gamma \ni x : t \\ \hline \Gamma \vdash x : t \end{array}$$

$$\begin{array}{c} \text{INT} \\ \hline \Gamma \vdash n : \text{int} \end{array}$$

$$\begin{array}{c} \text{PLUS} \\ \Gamma \vdash e1 : \text{int} \quad \Gamma \vdash e2 : \text{int} \\ \hline \Gamma \vdash e1 + e2 : \text{int} \end{array}$$

$$\begin{array}{c} \text{LET} \\ \Gamma \vdash e : t \quad \Gamma, x : t \vdash e' : t' \\ \hline \Gamma \vdash \text{let } x = e \text{ in } e' : t' \end{array}$$

$$\begin{array}{c} \text{FUN} \\ \Gamma, x : t \vdash e : t' \\ \hline \Gamma \vdash (\text{fun } (x : t) \rightarrow e) : t \rightarrow t' \end{array}$$

$$\begin{array}{c} \text{APP} \\ \Gamma \vdash e : t \rightarrow t' \quad \Gamma \vdash e' : t \\ \hline \Gamma \vdash e \ e' : t' \end{array}$$

Une présentation inductive du jugement de typage

On écrit $\Gamma \vdash e : t$ pour $(\Gamma, e, t) \in \mathcal{T}$.

$$\begin{array}{c} \text{VAR} \\ \Gamma \ni x : t \\ \hline \Gamma \vdash x : t \end{array}$$

$$\begin{array}{c} \text{INT} \\ \hline \Gamma \vdash n : \text{int} \end{array}$$

$$\begin{array}{c} \text{PLUS} \\ \Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int} \\ \hline \Gamma \vdash e_1 + e_2 : \text{int} \end{array}$$

$$\begin{array}{c} \text{LET} \\ \Gamma \vdash e : t \quad \Gamma, x : t \vdash e' : t' \\ \hline \Gamma \vdash \text{let } x = e \text{ in } e' : t' \end{array}$$

$$\begin{array}{c} \text{FUN} \\ \Gamma, x : t \vdash e : t' \\ \hline \Gamma \vdash (\text{fun } (x : t) \rightarrow e) : t \rightarrow t' \end{array}$$

$$\begin{array}{c} \text{APP} \\ \Gamma \vdash e : t \rightarrow t' \quad \Gamma \vdash e' : t \\ \hline \Gamma \vdash e \ e' : t' \end{array}$$

Définition (Dérivabilité)

Le jugement $\Gamma \vdash e : t$ est *dérivable* si on peut construire un arbre fini de racine $\Gamma \vdash e : t$ dont les nœuds sont étiquetés par les règles ci-dessus.

Présentation déclarative et présentation algorithmique

La présentation mathématique sous forme d'un jugement inductif :

- + est plus abstraite que le code (ne mentionne que les cas bien typés) ;
- + se prête au raisonnement par induction sur les dérivations ;
- $\pm$ ne décrit pas *a priori* un algorithme ! C'est une formulation *déclarative*.

Présentation déclarative et présentation algorithmique

La présentation mathématique sous forme d'un jugement inductif :

- + est plus abstraite que le code (ne mentionne que les cas bien typés) ;
- + se prête au raisonnement par induction sur les dérivations ;
- ± ne décrit pas *a priori* un algorithme ! C'est une formulation *déclarative*.

Proposition (Déterminisme du typage)

Si $\Gamma \vdash e : t_1$ et $\Gamma \vdash e : t_2$ alors $t_1 = t_2$.

Démonstration.

Par induction sur la dérivation de $\Gamma \vdash e : t_1$.



Présentation déclarative et présentation algorithmique

La présentation mathématique sous forme d'un jugement inductif :

- + est plus abstraite que le code (ne mentionne que les cas bien typés) ;
- + se prête au raisonnement par induction sur les dérivations ;
- $\pm$ ne décrit pas *a priori* un algorithme ! C'est une formulation *déclarative*.

Proposition (Déterminisme du typage)

Si $\Gamma \vdash e : t_1$ et $\Gamma \vdash e : t_2$ alors $t_1 = t_2$.

Démonstration.

Par induction sur la dérivation de $\Gamma \vdash e : t_1$.



Pouvez-vous donner un exemple de règle sûre qui invalide la propriété ?

Présentation déclarative et présentation algorithmique

La présentation mathématique sous forme d'un jugement inductif :

- + est plus abstraite que le code (ne mentionne que les cas bien typés) ;
- + se prête au raisonnement par induction sur les dérivations ;
- $\pm$ ne décrit pas *a priori* un algorithme ! C'est une formulation *déclarative*.

Proposition (Déterminisme du typage)

Si $\Gamma \vdash e : t_1$ et $\Gamma \vdash e : t_2$ alors $t_1 = t_2$.

Démonstration.

Par induction sur la dérivation de $\Gamma \vdash e : t_1$.



Pouvez-vous donner un exemple de règle sûre qui invalide la propriété ?

$$\frac{\Gamma, x : t \vdash e : t'}{\Gamma \vdash (\text{fun } x \rightarrow e) : t \rightarrow t'}$$

Exemple : la gestion des booléens

Exercice

Ajoutez l'équivalent des règles ci-dessous à synth_exp.

BOOL

$$\frac{}{\Gamma \vdash b : \text{bool}}$$

IF

$$\frac{\Gamma \vdash e1 : \text{bool} \quad \Gamma \vdash e2 : t \quad \Gamma \vdash e3 : t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 : t}$$

Exemple : la gestion des booléens

Exercice

Ajoutez l'équivalent des règles ci-dessous à `synth_exp`.

$$\begin{array}{c} \text{BOOL} \\ \hline \Gamma \vdash b : \text{bool} \end{array} \qquad \begin{array}{c} \text{IF} \\ \Gamma \vdash e1 : \text{bool} \quad \Gamma \vdash e2 : t \quad \Gamma \vdash e3 : t \\ \hline \Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 : t \end{array}$$

```
let rec synth_exp cx = function
```

```
...
```

```
| Bool _ -> TBool
```

```
| If (e1, e2, e3) ->
```

```
  check_exp cx TBool e1;
```

```
  let t = synth_exp cx e2 in (* choix arbitraire, comme pour Add *)
```

```
  check_exp cx t e3;
```

```
  t
```

De la spécification à l'implémentation

Comment implémenter un jugement inductif

$$\mathcal{J} \subseteq A_1 \times \cdots \times A_n$$

sous la forme d'une fonction récursive totale $A_1 \times \cdots \times A_n \rightarrow \mathbb{B}$?

Une recette informelle

- ➊ Identifier le *mode* de chaque composante A_i : entrée ou sortie ?
- ➋ Déterminer une entrée A_j qui décroît à chaque appel récursif.
- ➌ Regrouper les règles selon la forme de l'entrée A_j .

De la spécification à l'implémentation

Comment implémenter un jugement inductif

$$\mathcal{J} \subseteq A_1 \times \cdots \times A_n$$

sous la forme d'une fonction récursive totale $A_1 \times \cdots \times A_n \rightarrow \mathbb{B}$?

Une recette informelle

- ➊ Identifier le *mode* de chaque composante A_i : entrée ou sortie ?
- ➋ Déterminer une entrée A_j qui décroît à chaque appel récursif.
- ➌ Regrouper les règles selon la forme de l'entrée A_j .

Cette recette fonctionne pour les jugements d'évaluation (jalon 2) et de typage (jalon 3), pas pour le jugement d'acceptation d'un mot par une grammaire (cf. cours d'analyse syntaxique).

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Des limites du typage monodirectionnel

Notre vérificateur de types requiert beaucoup d'annotations. Par exemple :

```
let g = fun (f : int -> int) -> fun (x : int) -> f (x + 1) in  
g (fun (y : int) -> y + 1)
```

L'annotation sur `y` est clairement redondante connaissant le type de `g`.

Des limites du typage monodirectionnel

Notre vérificateur de types requiert beaucoup d'annotations. Par exemple :

```
let g = fun (f : int -> int) -> fun (x : int) -> f (x + 1) in  
g (fun (y : int) -> y + 1)
```

L'annotation sur *y* est clairement redondante connaissant le type de *g*.

Deux approches parmi d'autres

- ❶ L'**inférence de type** permet d'éviter toute annotation dans le code ci-dessus via un processus global de résolution de contrainte.
- ❷ Le **typage bidirectionnel** permet d'éviter certaines annotations par un processus local de propagation d'information.

Des limites du typage monodirectionnel

Notre vérificateur de types requiert beaucoup d'annotations. Par exemple :

```
let g = fun (f : int -> int) -> fun (x : int) -> f (x + 1) in  
g (fun (y : int) -> y + 1)
```

L'annotation sur y est clairement redondante connaissant le type de g.

Deux approches parmi d'autres

- 1 L'**inférence de type** permet d'éviter toute annotation dans le code ci-dessus via un processus global de résolution de contrainte.
- 2 Le **typage bidirectionnel** permet d'éviter certaines annotations par un processus local de propagation d'information.

Étudions la deuxième approche, à implémenter pour Hopix dans le jalon 3.

Retour au vérificateur de types

```
let rec synth_exp cx = function
| Var x -> lookup x cx | Int _ -> TInt
| Add (e1, e2) -> check_exp cx TInt e1; check_exp cx TInt e2; TInt
| App (e1, e2) -> app_types cx (synth_exp cx e1) e2
| Let (e, b) -> synth_bound1 cx (synth_exp cx e) b
| Fun (dom, b) -> TFun (dom, synth_bound1 cx dom b)

and check_exp cx expected e =
  let given = synth_exp cx e in
  if given <> expected then type_mismatch ~expected ~given e
```

Retour au vérificateur de types

```
let rec synth_exp cx = function
| Var x -> lookup x cx | Int _ -> TInt
| Add (e1, e2) -> check_exp cx TInt e1; check_exp cx TInt e2; TInt
| App (e1, e2) -> app_types cx (synth_exp cx e1) e2
| Let (e, b) -> synth_bound1 cx (synth_exp cx e) b
| Fun (dom, b) -> TFun (dom, synth_bound1 cx dom b)

and check_exp cx expected e =
  let given = synth_exp cx e in
  if given <> expected then type_mismatch ~expected ~given e
```

Observations

- La fonction `check_exp` n'exploite pas le type `expected`.
- La notion d'annotation de typage a du sens au delà des paramètres.

```
type t = ... | Fun of bound1 | Annot of t * ty
```

Un algorithme de typage bidirectionnel

```
let rec synth_exp cx = function
| Var x -> lookup x cx | Int _ -> TInt
| Add (e1, e2) -> check_exp cx TInt e1; check_exp cx TInt e2; TInt
| App (e1, e2) -> app_types cx (synth_exp cx e1) e2
| Let (e, b) -> synth_bound1 (synth_exp cx e) b
| Annot (e, expected) -> check_exp cx expected e; expected
| (Fun _) as e -> cannot_synthesize e

and check_exp cx expected e = match e with
| Var _ | Int _ | Add _ | App _ | Annot _ ->
  let given = synth_exp cx e in
  if given <> expected then type_mismatch ~expected ~given e
| Let (e, b) ->
  check_bound1 cx (synth_exp cx e) expected b
| Fun b ->
  begin match expected with
  | TFun (dom, cod) -> check_bound1 cx dom cod b
  | _ -> type_mismatch_expected_fun expected
  end
```

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Deux jugements inductifs pour le typage bidirectionnel

SYNTHVAR

$$\frac{\Gamma \ni x : t}{\Gamma \vdash x \Rightarrow t}$$

SYNTHINT

$$\frac{}{\Gamma \vdash n \Rightarrow \text{int}}$$

SYNTHPLUS

$$\frac{\Gamma \vdash e1 \Leftarrow \text{int} \quad \Gamma \vdash e2 \Leftarrow \text{int}}{\Gamma \vdash e1 + e2 \Rightarrow \text{int}}$$

SYNTHLET

$$\frac{\Gamma \vdash e \Rightarrow t \quad \Gamma, x : t \vdash e' \Rightarrow t'}{\Gamma \vdash \text{let } x = e \text{ in } e' \Rightarrow t'}$$

SYNTHAPP

$$\frac{\Gamma \vdash e \Rightarrow t \rightarrow t' \quad \Gamma \vdash e' \Leftarrow t}{\Gamma \vdash e e' \Rightarrow t'}$$

SYNTHCHECK

$$\frac{\Gamma \vdash e \Leftarrow t}{\Gamma \vdash (e : t) \Rightarrow t}$$

CHECKSYNTH

$$\frac{\Gamma \vdash e \Rightarrow t}{\Gamma \vdash e \Leftarrow t}$$

CHECKFUN

$$\frac{\Gamma, x : t \vdash e \Leftarrow t'}{\Gamma \vdash (\text{fun } x \rightarrow e) \Leftarrow t \rightarrow t'}$$

CHECKLET

$$\frac{\Gamma \vdash e \Rightarrow t \quad \Gamma, x : t \vdash e' \Leftarrow t'}{\Gamma \vdash \text{let } x = e \text{ in } e' \Leftarrow t'}$$

Quels résultats ?

On note $\Gamma \vdash e : t$ pour le jugement *déclaratif* muni de la règle ci-dessous.

$$\frac{\Gamma, x : t \vdash e : t'}{\Gamma \vdash (\text{fun } x \rightarrow e) : t \rightarrow t'}$$

On écrit $\lfloor e \rfloor$ pour l'expression e dont on a effacé toutes les annotations.

Quels résultats ?

On note $\Gamma \vdash e : t$ pour le jugement *déclaratif* muni de la règle ci-dessous.

$$\frac{\Gamma, x : t \vdash e : t'}{\Gamma \vdash (\text{fun } x \rightarrow e) : t \rightarrow t'}$$

On écrit $\lfloor e \rfloor$ pour l'expression e dont on a effacé toutes les annotations.

Théorème (Correction du typage bidirectionnel)

Si $\Gamma \vdash e \Rightarrow t$ ou $\Gamma \vdash e \Leftarrow t$ alors $\Gamma \vdash \lfloor e \rfloor : t$.

Quels résultats ?

On note $\Gamma \vdash e : t$ pour le jugement *déclaratif* muni de la règle ci-dessous.

$$\frac{\Gamma, x : t \vdash e : t'}{\Gamma \vdash (\text{fun } x \rightarrow e) : t \rightarrow t'}$$

On écrit $\lfloor e \rfloor$ pour l'expression e dont on a effacé toutes les annotations.

Théorème (Correction du typage bidirectionnel)

Si $\Gamma \vdash e \Rightarrow t$ ou $\Gamma \vdash e \Leftarrow t$ alors $\Gamma \vdash \lfloor e \rfloor : t$.

Théorème (Complétude partielle du typage bidirectionnel)

Si $\Gamma \vdash e : t$ alors :

- 1 il existe $e_1 \in \lfloor e \rfloor^{-1}$ tel que $\Gamma \vdash e_1 \Rightarrow t$,
- 2 il existe $e_2 \in \lfloor e \rfloor^{-1}$ tel que $\Gamma \vdash e_2 \Leftarrow t$.

Gestion des expressions booléennes

Exercice

Étendre le typage bidirectionnel aux conditionnelles.

Gestion des expressions booléennes

Exercice

Étendre le typage bidirectionnel aux conditionnelles.

CHECKIF ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Leftarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Leftarrow t}$$

Gestion des expressions booléennes

Exercice

Étendre le typage bidirectionnel aux conditionnelles.

CHECKIF ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Leftarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Leftarrow t}$$

SYNTHIF1 ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Rightarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow t}$$

Gestion des expressions booléennes

Exercice

Étendre le typage bidirectionnel aux conditionnelles.

CHECKIF ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Leftarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Leftarrow t}$$

SYNTHIF1 ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Rightarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow t}$$

SYNTHIF2 ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Leftarrow t \quad \Gamma \vdash e3 \Rightarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow t}$$

Gestion des expressions booléennes

Exercice

Étendre le typage bidirectionnel aux conditionnelles.

CHECKIF ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Leftarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Leftarrow t}$$

SYNTHIF1 ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Rightarrow t \quad \Gamma \vdash e3 \Leftarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow t}$$

SYNTHIF2 ?

$$\frac{\Gamma \vdash e1 \Leftarrow \text{bool} \quad \Gamma \vdash e2 \Leftarrow t \quad \Gamma \vdash e3 \Rightarrow t}{\Gamma \vdash \text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow t}$$

La première règle se généralise aux types sommes n -aires.

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Bilan au sujet du typage bidirectionnel

L'expression ci-dessous est de type `int -> int` d'après `synth_exp`.

```
let g = (fun f -> fun x -> f (x + 1)) : (int -> int) -> int -> int in
g (fun y -> y + 1)
```

On peut trouver cette quantité d'annotations raisonnable.

Bilan

- Une technique simple pour réduire la quantité d'annotations.
 - Pas aussi flexible que l'inférence de types à la ML. Complétude partielle.
 - Compatible avec des systèmes bien plus expressifs, e.g., sous-typage.
- Parfois délicat de choisir une direction pour une construction donnée.
 - Pour la conditionnelle, le choix de `CHECKIF` est le plus naturel.

Bilan au sujet du typage bidirectionnel

L'expression ci-dessous est de type `int -> int` d'après `synth_exp`.

```
let g = (fun f -> fun x -> f (x + 1)) : (int -> int) -> int -> int in  
g (fun y -> y + 1)
```

On peut trouver cette quantité d'annotations raisonnable.

Bilan

- Une technique simple pour réduire la quantité d'annotations.
 - Pas aussi flexible que l'inférence de types à la ML. Complétude partielle.
 - Compatible avec des systèmes bien plus expressifs, e.g., sous-typage.
- Parfois délicat de choisir une direction pour une construction donnée.
 - Pour la conditionnelle, le choix de `CHECKIF` est le plus naturel.

Le typage bidirectionnel s'étend facilement pour traiter types algébriques, enregistrements, références, etc. Que manque-t-il pour Hopix ?

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Pourquoi le polymorphisme ?

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow \text{int} \rightarrow \text{int}$

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow (\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})$

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{int}) \rightarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{int})$

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow t \rightarrow t$ quel que soit t

Pourquoi le polymorphisme ?

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow \text{int} \rightarrow \text{int}$

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow (\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})$

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{int}) \rightarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{int})$

$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow t \rightarrow t$ quel que soit t

Un défaut d'expressivité

Aucun type (**objet**) ne représente la famille (**sujet**) de types $(t \rightarrow t)_{t \in Ty}$.

Pourquoi le polymorphisme ?

$$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow \text{int} \rightarrow \text{int}$$
$$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow (\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})$$
$$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{int}) \rightarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{int})$$
$$\Gamma \vdash \text{fun } x \rightarrow x \Leftarrow t \rightarrow t \text{ quel que soit } t$$

Un défaut d'expressivité

Aucun type (**objet**) ne représente la famille (**sujet**) de types $(t \rightarrow t)_{t \in T_y}$.

La quantification universelle et le polymorphisme

On peut représenter cette famille de types par le type $\forall \alpha. \alpha \rightarrow \alpha$.

On aura donc, dans l'exemple, $\Gamma \vdash \text{fun } x \rightarrow x : \forall \alpha. \alpha \rightarrow \alpha$.

Le typage polymorphe

$$\frac{?}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\Gamma \vdash e' : \forall \alpha. t}{?}$$

Quelles intuitions ?

Le typage polymorphe

$$\frac{?}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\Gamma \vdash e' : \forall \alpha. t}{?}$$

Quelles intuitions ?

- L'expression e doit se comporter *uniformément* vis-à-vis de α pour être de type polymorphe. Exemple canonique : `List.length`.

Le typage polymorphe

$$\frac{?}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\Gamma \vdash e' : \forall \alpha. t}{?}$$

Quelles intuitions ?

- L'expression e doit se comporter *uniformément* vis-à-vis de α pour être de type polymorphe. Exemple canonique : `List.length`.
- L'expression e' peut être utilisée avec n'importe quel type t où α a été remplacé par un t' quelconque... ou presque.

Le typage polymorphe

$$\frac{?}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\Gamma \vdash e' : \forall \alpha. t}{?}$$

Quelles intuitions ?

- L'expression e doit se comporter *uniformément* vis-à-vis de α pour être de type polymorphe. Exemple canonique : `List.length`.
- L'expression e' peut être utilisée avec n'importe quel type t où α a été remplacé par un t' quelconque... ou presque.

On peut essayer capturer ces intuitions, déjà via un jugement déclaratif.

Règles déclaratives naïves pour le polymorphisme

GENERALIZE

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

INstantiate

$$\frac{\Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

Règles déclaratives naïves pour le polymorphisme

$$\text{GENERALIZE} \quad \frac{\Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

$$\text{INSTANTIATE} \quad \frac{\Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

$$\frac{}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta} \text{GENERALIZE}^2$$

Règles déclaratives naïves pour le polymorphisme

GENERALIZE

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

INstantiate

$$\frac{\Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

$$\frac{\frac{}{\bullet \vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta} \text{FUN}}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta} \text{GENERALIZE}^2$$

Règles déclaratives naïves pour le polymorphisme

GENERALIZE

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

INstantiate

$$\frac{\Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

$$\frac{\frac{\frac{}{x : \alpha \vdash x : \beta} \text{INstantiate}}{\bullet \vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta} \text{FUN}}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta} \text{GENERALIZE}^2$$

Règles déclaratives naïves pour le polymorphisme

GENERALIZE

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

INstantiate

$$\frac{\Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

$$\frac{\frac{\frac{\frac{}{x : \alpha \vdash x : \forall \alpha. \alpha}}{x : \alpha \vdash x : \beta}}{\bullet \vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta}}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta}$$

GENERALIZE
INstantiate
FUN
GENERALIZE²

Règles déclaratives naïves pour le polymorphisme

GENERALIZE

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

INstantiate

$$\frac{\Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

$$\frac{\frac{\frac{x : \alpha \ni x : \alpha}{x : \alpha \vdash x : \alpha} \text{VAR}}{x : \alpha \vdash x : \forall \alpha. \alpha} \text{GENERALIZE}}{x : \alpha \vdash x : \beta} \text{INstantiate}$$
$$\frac{\bullet \vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta} \text{FUN}$$
$$\frac{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta} \text{GENERALIZE}^2$$

Règles déclaratives naïves pour le polymorphisme

$$\frac{\text{GENERALIZE} \quad \Gamma \vdash e : t}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\text{INSTANTIATE} \quad \Gamma \vdash e : \forall \alpha. t}{\Gamma \vdash e : t[\alpha \backslash t']}$$

Ces règles sont-elles satisfaisantes ?

$$\frac{\frac{\frac{x : \alpha \ni x : \alpha}{x : \alpha \vdash x : \alpha} \text{VAR}}{x : \alpha \vdash x : \forall \alpha. \alpha} \text{GENERALIZE}}{x : \alpha \vdash x : \beta} \text{INSTANTIATE} \quad \frac{\bullet \vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta} \text{FUN}$$

$\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta$ GENERALIZE²

Ces règles sont donc **fausses**, invalidant la sûreté du typage.

"toto" ^ (fun x -> x) 42

Analyse du problème

$$\frac{\frac{\frac{x : \alpha \ni x : \alpha}{x : \alpha \vdash x : \alpha}}{x : \alpha \vdash x : \forall \alpha. \alpha}}{x : \alpha \vdash x : \beta}$$
$$\frac{\bullet \vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta}$$

Analyse du problème

$$\frac{\frac{\frac{x : \alpha \ni x : \alpha}{x : \alpha \vdash x : \alpha}}{x : \alpha \vdash x : \forall \alpha. \alpha}}{x : \alpha \vdash x : \beta}$$

- $\vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta$

$$\frac{}{\bullet \vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta}$$

Il y a une **confusion** au moment de la généralisation. En particulier, cette dérivation est incompatible avec le renommage des variables de types liées.

Analyse du problème

$$\frac{\frac{\frac{x : \alpha \ni x : \alpha}{x : \alpha \vdash x : \alpha}}{x : \alpha \vdash x : \forall \alpha. \alpha}}{x : \alpha \vdash x : \beta}$$

• $\vdash \text{fun } x \rightarrow x : \alpha \rightarrow \beta$

• $\vdash \text{fun } x \rightarrow x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta$

Il y a une **confusion** au moment de la généralisation. En particulier, cette dérivation est incompatible avec le renommage des variables de types liées.

mal typé!

$$\frac{\frac{\frac{x : \alpha \vdash x : \gamma}{x : \alpha \vdash x : \forall \gamma. \gamma}}{x : \alpha \vdash x : \beta}}$$

Le polymorphisme en style déclaratif et implicite

- On considère les types à renommage des variables de types liées près.
- Les contextes déclarent désormais les variables de types.
 - Une variable de type donnée apparaît au plus une fois dans un contexte.

$$t ::= \text{int} \mid \text{bool} \mid t \rightarrow t' \mid \alpha \mid \forall \alpha. t$$

GENERALIZE

$$\frac{\Gamma, \alpha \vdash e : t \quad \Gamma \not\ni \alpha}{\Gamma \vdash e : \forall \alpha. t}$$

INSTANTIATE

$$\frac{\Gamma \vdash e : \forall \alpha. t \quad \Gamma \vdash t' \text{ type}}{\Gamma \vdash e : t[\alpha \backslash t']}$$

$$\frac{\Gamma \ni \alpha}{\Gamma \vdash \alpha \text{ type}}$$

$$\frac{}{\Gamma \vdash \text{int type}}$$

...

$$\frac{\Gamma \vdash t \text{ type} \quad \Gamma \vdash t' \text{ type}}{\Gamma \vdash t \rightarrow t' \text{ type}}$$

Bilan

$$\frac{\text{GENERALIZE} \quad \Gamma, \alpha \vdash e : t \quad \Gamma \not\vdash \alpha}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\text{INSTANTIATE} \quad \Gamma \vdash e : \forall \alpha. t \quad \Gamma \vdash t' \text{ type}}{\Gamma \vdash e : t[\alpha \backslash t']}$$

- Ces règles sont simples et satisfaisantes du point de vue structurel.
 - Règle d'introduction vs. règle d'élimination.
- Elles garantissent la sûreté du typage, au sens précédent.
- Elles sont impossibles à implémenter tel quel ! (WELLS 1994)

Bilan

$$\frac{\text{GENERALIZE} \quad \Gamma, \alpha \vdash e : t \quad \Gamma \not\vdash \alpha}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\text{INSTANTIATE} \quad \Gamma \vdash e : \forall \alpha. t \quad \Gamma \vdash t' \text{ type}}{\Gamma \vdash e : t[\alpha \backslash t']}$$

- Ces règles sont simples et satisfaisantes du point de vue structurel.
 - Règle d'introduction vs. règle d'élimination.
- Elles garantissent la sûreté du typage, au sens précédent.
- Elles sont impossibles à implémenter tel quel ! (WELLS 1994)

Pour la culture : le système F, le cœur théorique du polymorphisme

- Le langage restreint aux règles traitant fonctions et polymorphisme.
- Tous ses programmes terminent. Preuve très difficile (GIRARD 1972).

Bilan

$$\frac{\text{GENERALIZE} \quad \Gamma, \alpha \vdash e : t \quad \Gamma \not\vdash \alpha}{\Gamma \vdash e : \forall \alpha. t}$$

$$\frac{\text{INSTANTIATE} \quad \Gamma \vdash e : \forall \alpha. t \quad \Gamma \vdash t' \text{ type}}{\Gamma \vdash e : t[\alpha \backslash t']}$$

- Ces règles sont simples et satisfaisantes du point de vue structurel.
 - Règle d'introduction vs. règle d'élimination.
- Elles garantissent la sûreté du typage, au sens précédent.
- Elles sont impossibles à implémenter tel quel ! (WELLS 1994)

Pour la culture : le système F, le cœur théorique du polymorphisme

- Le langage restreint aux règles traitant fonctions et polymorphisme.
- Tous ses programmes terminent. Preuve très difficile (GIRARD 1972).

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Deux solutions possibles

Solution 1 : le polymorphisme explicite

On **écrit** explicitement les points de généralisation et instantiation.

$$\begin{array}{c} \text{GENERALIZE} \\ \hline \Gamma, \alpha \vdash e : t \quad \Gamma \not\vdash \alpha \\ \hline \Gamma \vdash \text{fun } (\text{type } \alpha) \rightarrow e : \forall \alpha. t \end{array} \qquad \begin{array}{c} \text{INSTANTIATE} \\ \hline \Gamma \vdash e : \forall \alpha. t \quad \Gamma \vdash t' \text{ type} \\ \hline \Gamma \vdash e[t'] : t[\alpha \setminus t'] \end{array}$$

Solution 2 : le polymorphisme implicite à la ML

On **restreint** quantification et points d'instanciation du polymorphisme.

$$\begin{aligned} t &::= \text{int} \mid \text{bool} \mid t \rightarrow t' \\ s &::= \forall \alpha_1, \dots, \alpha_n. t \end{aligned}$$

On généralise aux définitions (**let**), instancie aux occurrences de variables.

Le polymorphisme à la ML, formulé déclarativement

LET

$$\frac{\Gamma \vdash e : t \quad \Gamma, x : Gen_{\Gamma}(t) \vdash e' : t'}{\Gamma \vdash \text{let } x = e \text{ in } e' : t'}$$

VAR

$$\frac{(\Gamma \vdash t_i \text{ type})_{i \in [1, n]} \quad \Gamma \ni x : \forall(\alpha_1, \dots, \alpha_n). t}{\Gamma \vdash x : t[\alpha_1 \setminus t_1, \dots, \alpha_n \setminus t_n]}$$

$$Gen_{\Gamma}(t) \triangleq \forall(\alpha_1, \dots, \alpha_n). t \text{ où } \{\alpha_1, \dots, \alpha_n\} = FTV(t) \setminus FTV(\Gamma)$$

Le polymorphisme à la ML, formulé déclarativement

LET

$$\frac{\begin{array}{c} \Gamma \vdash e : t \\ \Gamma, x : Gen_{\Gamma}(t) \vdash e' : t' \end{array}}{\Gamma \vdash \text{let } x = e \text{ in } e' : t'}$$

VAR

$$\frac{\begin{array}{c} (\Gamma \vdash t_i \text{ type})_{i \in [1, n]} \\ \Gamma \ni x : \forall(\alpha_1, \dots, \alpha_n). t \end{array}}{\Gamma \vdash x : t[\alpha_1 \setminus t_1, \dots, \alpha_n \setminus t_n]}$$

$$Gen_{\Gamma}(t) \triangleq \forall(\alpha_1, \dots, \alpha_n). t \text{ où } \{\alpha_1, \dots, \alpha_n\} = FTV(t) \setminus FTV(\Gamma)$$

- Le jugement est déclaratif. Si la règle LET est algorithmique, ce n'est pas du tout le cas pour règle VAR où il faut deviner les types t_i .

Le polymorphisme à la ML, formulé déclarativement

LET

$$\frac{\begin{array}{c} \Gamma \vdash e : t \\ \Gamma, x : Gen_{\Gamma}(t) \vdash e' : t' \end{array}}{\Gamma \vdash \text{let } x = e \text{ in } e' : t'}$$

VAR

$$\frac{\begin{array}{c} (\Gamma \vdash t_i \text{ type})_{i \in [1, n]} \\ \Gamma \ni x : \forall(\alpha_1, \dots, \alpha_n). t \end{array}}{\Gamma \vdash x : t[\alpha_1 \setminus t_1, \dots, \alpha_n \setminus t_n]}$$

$$Gen_{\Gamma}(t) \triangleq \forall(\alpha_1, \dots, \alpha_n). t \text{ où } \{\alpha_1, \dots, \alpha_n\} = FTV(t) \setminus FTV(\Gamma)$$

- Le jugement est déclaratif. Si la règle LET est algorithmique, ce n'est pas du tout le cas pour règle VAR où il faut deviner les types t_i .
- L'**algorithme W** de DAMAS et MILNER (1982) est une formulation algorithmique qui sert de base de l'inférence de types à la OCaml.
 - (Dépasse le cadre de cette séance.)

Splendeurs et misères de l'inférence de types à la ML

Splendeurs et misères de l'inférence de types à la ML

Théorème (Correction de l'algorithme W)

Si $\Gamma \vdash e \Rightarrow_W t$ alors $\Gamma \vdash e : t$.

Splendeurs et misères de l'inférence de types à la ML

Théorème (Correction de l'algorithme W)

Si $\Gamma \vdash e \Rightarrow_W t$ alors $\Gamma \vdash e : t$.

Théorème (Complétude de l'algorithme W)

Si $\Gamma \vdash e : t$ alors :

- *il existe t' tel que $\Gamma \vdash e \Rightarrow_W t'$, et*
- *le type t est une instance (spécialisation) de t' .*

Splendeurs et misères de l'inférence de types à la ML

Théorème (Correction de l'algorithme W)

Si $\Gamma \vdash e \Rightarrow_W t$ alors $\Gamma \vdash e : t$.

Théorème (Complétude de l'algorithme W)

Si $\Gamma \vdash e : t$ alors :

- *il existe t' tel que $\Gamma \vdash e \Rightarrow_W t'$, et*
- *le type t est une instance (spécialisation) de t' .*

Splendeurs

L'algorithme W calcule le type principal (le plus général) de l'expression.

Splendeurs et misères de l'inférence de types à la ML

Théorème (Correction de l'algorithme W)

Si $\Gamma \vdash e \Rightarrow_W t$ alors $\Gamma \vdash e : t$.

Théorème (Complétude de l'algorithme W)

Si $\Gamma \vdash e : t$ alors :

- *il existe t' tel que $\Gamma \vdash e \Rightarrow_W t'$, et*
- *le type t est une instance (spécialisation) de t' .*

Splendeurs

L'algorithme W calcule le type principal (le plus général) de l'expression.

Misères

L'expression (**fun** $f \rightarrow (f\ 1, f\ \text{"toto"})$) est mal typée en ML.

Plan

1 Introduction

2 Les types simples

- Une présentation informatique
- Une présentation mathématique

3 Le typage bidirectionnel

- Introduction et présentation informatique
- Présentation mathématique
- Bilan

4 Le polymorphisme

- Introduction et polymorphisme déclaratif
- Le polymorphisme à la ML

5 Conclusion

Conclusion

Le typage est une *analyse statique* qui :

- garantit l'absence de mauvais comportements à l'exécution ;
- est syntaxique, intégrée au langage, modulaire ;

$$\frac{\text{SUBSTITUTION} \quad \Gamma, x : t_1 \vdash e : t_2 \quad \Gamma \vdash e' : t_1}{\Gamma \vdash e[x \setminus e'] : t_2}$$

- s'étend à des langages très riches (avec objets, modules, etc.) ;
- est souvent exploitée par le générateur de code.

Jalon 3

Vérification bidirectionnelle pour Hopix avec des annotations explicites.

Bibliographie



DAMAS, Luis et Robin MILNER (1982). “Principal type-schemes for functional programs”. In : *Proceedings of the 9th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '82*. DOI : 10.1145/582153.582176. URL : https://web.cs.wpi.edu/~cs4536/c12/milner-damas_principal_types.pdf.



GIRARD, Jean-Yves (1972). “Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur”. Université Paris 7.



WELLS, J. B. (1994). “Typability and type checking in the second-order lambda-calculus are equivalent and undecidable”. In : *Proceedings Ninth Annual IEEE Symposium on Logic in Computer Science*. DOI : 10.1109/lics.1994.316068. URL : <http://dx.doi.org/10.1109/lics.1994.316068>.